



UNIVERSIDAD  
DE GRANADA

GENERACIÓN DE DATOS SINTÉTICA A PARTIR DE UN  
SISTEMA IOT DE DETECCIÓN DE VEHÍCULOS UTILIZANDO  
MODELOS TRANSFORMER CON ATENCIÓN EN PYTHON

RICARDO RUIZ FERNÁNDEZ DE ALBA

Trabajo Fin de Grado

Doble Grado en Ingeniería Informática y Matemáticas

**Tutores**

María Bermúdez Edo

Daniel Bolaños

FACULTAD DE CIENCIAS

E.T.S. INGENIERÍAS INFORMÁTICA Y DE TELECOMUNICACIÓN

*Granada, a 7 de abril de 2025*

---

## ÍNDICE GENERAL

---

---

## RESUMEN

---

---

## INTRODUCCIÓN Y OBJETIVOS

---

### 1.1 INTRODUCCIÓN

#### 1.1.1 *Contextualización*

La Inteligencia Artificial (IA), impulsada por el avance en algoritmos de aprendizaje automático ha experimentado un rápido desarrollo debido a su reciente viabilidad de ejecución en hardware y la creciente disponibilidad de datos.

Sus fundamentos matemáticos, incluyendo el análisis, la probabilidad y el álgebra lineal que conforman la teoría del aprendizaje estadístico son esenciales para construir modelos que aprenden de los datos y realizan predicciones.

Como aplicación particular se encuentra la capacidad de estos algoritmos para identificar patrones en datos generados por las tecnologías de Internet Industrial de las Cosas (IoT), basadas en redes de sensores y dispositivos conectados a Internet que recopilan y transmiten datos en tiempo real.

#### 1.1.2 *Descripción del problema*

El marco de este trabajo está relacionado con un proyecto de investigación de un sistema IoT implementado en la región de Poqueira, en la Alpujarra granadina ?. El sistema IoT ha procesado la información recogida por los sensores visuales y la ha exportado a un conjunto de datos con múltiples características.

El entrenamiento de modelos de Aprendizaje Automático a partir de estos datos podría predecir variables que mejorasen la gestión de la ocupación de la zona, promoviendo así una mayor sostenibilidad en la comunidad local.

Sin embargo, la complejidad de la información obtenida, la presencia de ruido y la falta de datos etiquetados, dificultan la tarea. En este contexto, la generación de datos sintéticos se presenta como solución, pues permite añadir al conjunto de datos

original muestras generadas artificialmente que simulan las características de los datos reales. Esta técnica facilita el entrenamiento de modelos más robustos y precisos, abordando la escasez de etiquetas y mejorando la precisión de los modelos.

Aunque la literatura existente ha utilizado diversos Modelos de Generación Profunda (DGMs), incluyendo Redes Generativas Antagónicas (GANs) y Modelos de Autoencoders Variacionales (VAEs) ?, en este proyecto nos centraremos en los Modelos Transformers con Atención, pues su capacidad de modelar dependencias a largo plazo en los datos, los presenta como una alternativa prometedora.

### 1.1.3 Estructura del trabajo

Este trabajo se organiza en cinco partes, cada una de las cuales se divide en capítulos que abordan diferentes aspectos del problema y su solución. La estructura sigue las directrices establecidas para los trabajos de fin de grado.

- **Primera parte:** Presenta la introducción y los objetivos del proyecto.
- **Segunda parte:** Detalla los fundamentos matemáticos y teóricos del aprendizaje automático que sustentan el trabajo.
- **Tercera parte:** Se centra en el análisis de los modelos Transformer y perceptrones multicapa, explorando sus fundamentos matemáticos y arquitectura.
- **Cuarta parte:** Describe el desarrollo del proyecto y los experimentos realizados, incluyendo detalles sobre el problema, el software utilizado, el preprocesamiento de datos y las métricas de evaluación.
- **Quinta parte:** Presenta las conclusiones y reflexiones finales sobre el trabajo, así como recomendaciones para futuras investigaciones.

### 1.1.4 Herramientas Matemáticas e Informáticas

Para el desarrollo de este trabajo se utilizarán herramientas matemáticas e informáticas clave. Los fundamentos matemáticos incluyen análisis (diferenciabilidad), probabilidad y estadística para el manejo de la incertidumbre, y álgebra lineal, esenciales en el diseño y optimización de modelos de *machine learning*.

La implementación se llevará a cabo en Python, utilizando bibliotecas como *Numpy*, *Pandas*, y *Matplotlib* para el análisis de datos, y *Pypots* para el entrenamiento de los modelos *Transformers*.

### 1.1.5 *Bibliografía Fundamental*

Para la elaboración del Capítulo 3, Teoría del Aprendizaje Estadístico se ha destacamos el uso del libro Learning from Data ?.

## 1.2 OBJETIVOS

### 1.2.1 *Objetivo General*

El objetivo general de este trabajo es revisar los fundamentos matemáticos del aprendizaje automático y aplicar este conocimiento en la implementación de modelos generativos, específicamente utilizando arquitecturas Transformer.

### 1.2.2 *Parte de Matemáticas Específicos*

1. **Estudiar los fundamentos matemáticos del aprendizaje automático:** Revisar y comprender los conceptos matemáticos y teóricos que sustentan el aprendizaje automático, incluyendo teoría sobre cotas de generalización, optimización y regularización.

El objetivo 1 de matemáticas se abordará en el Capítulo 2.

### 1.2.3 *Parte de Informática*

1. **Definir la arquitectura Transformer. Modelo SAITS** Definir Encoder, Decoder, FFN, mecanismo de atención y como la arquitectura transformers los combina.
2. **Descripción de los conjuntos de datos. EDA** Llevar a cabo una descripción del conjunto de dato obtenido del sistema IoT de detección de vehículos. Realizar un análisis exploratorio de los datos.
3. **Entrenamiento mediante la biblioteca PyPOTS. Rendimiento** Evaluar el rendimiento y efectividad del modelo Transformer con atención a partir de métricas de evaluación, comparándolo con otros enfoques de modelado generativo.

El objetivo 1 de Informática se abordará en el Capítulo 3, el objetivo 2 en el Capítulo 4 y el objetivo 3 en el Capítulo 5.

---

## FUNDAMENTOS MATEMÁTICOS DEL APRENDIZAJE AUTOMÁTICO

---

Taxonomía de Machine Learning ->Supervisado, no Supervisado

Estadístico ->Imputación

---

## DEL TRANSFORMER A SAITS: ARQUITECTURA Y APLICACIÓN

---

### 3.0.1 *Introducción a los Modelos Transformer*

Los modelos Transformer irrumpieron en el panorama del aprendizaje automático como una auténtica disrupción, marcando un antes y un después en el procesamiento de secuencias, especialmente en el ámbito del lenguaje natural (NLP). Su concepción representó un avance paradigmático, superando las limitaciones inherentes a las arquitecturas recurrentes (RNNs) y convolucionales (CNNs) que hasta entonces dominaban el campo. La esencia de su revolución reside en dos pilares fundamentales: su capacidad intrínseca para el **procesamiento en paralelo** y la introducción de un mecanismo de atención sumamente potente y flexible. Mientras que las RNNs procesaban la información de manera secuencial, limitando la eficiencia y la capacidad de capturar dependencias a largo alcance, los Transformers abordan la secuencia de entrada como un todo, operando simultáneamente sobre todos sus elementos. Este enfoque, orquestado por el mecanismo de atención, dota al modelo de la habilidad de discernir y ponderar la importancia relativa de cada parte de la entrada con respecto a las demás. En lugar de procesar la información paso a paso, el Transformer evalúa la relevancia contextual de cada elemento, optimizando el flujo de información y, en última instancia, potenciando significativamente su capacidad de aprendizaje y comprensión. Este capítulo se adentra en el corazón de la arquitectura Transformer, desglosando sus componentes esenciales y trazando un camino que culmina en SAITS (Self-Attention-based Imputation for Time Series), un modelo especializado que adapta la potencia del Transformer a la imputación de datos faltantes en series temporales. A través de este recorrido, demostraremos la versatilidad y el alcance de esta arquitectura, extendiéndose mucho más allá de sus orígenes en el procesamiento del lenguaje natural.

### 3.0.2 *Mecanismo de Atención Escalonada (Scaled Dot-Product Attention)*

El mecanismo de atención constituye el núcleo operativo y la innovación distintiva de los Transformers, siendo el principal artífice de su excepcional rendimiento. Den-



tro del espectro de mecanismos de atención existentes, la **atención escalonada por producto punto** (Scaled Dot-Product Attention) emerge como la piedra angular, el bloque constructivo esencial. Este ingenioso mecanismo habilita al modelo para discernir y cuantificar la similitud o relevancia entre diferentes posiciones dentro de la secuencia de entrada. De esta forma, determina con precisión qué fragmentos de la información entrante merecen una mayor ponderación al procesar cada elemento en particular. En términos prácticos, la atención confiere al modelo la capacidad de enfocar su "mirada" selectivamente sobre la información más crucial y pertinente en cada instante del procesamiento, emulando, en cierto modo, la atención selectiva humana.

### 3.0.2.1 *Matrices de Consulta, Clave y Valor (Query, Key, Value)*

La orquestación de la atención escalonada se basa en la interacción estratégica de tres matrices fundamentales: **Consulta (Query)**, **Clave (Key)** y **Valor (Value)**. Estas matrices, representadas convencionalmente como  $Q$ ,  $K$  y  $V$  respectivamente, no son entidades independientes, sino que se derivan de la secuencia de entrada original a través de proyecciones lineales individuales y dedicadas. Para comprender mejor su función, podemos recurrir a una analogía conceptual:

- **Consulta (Q):** La matriz de Consulta ( $Q$ ) personifica la pregunta o la necesidad de información del elemento que se está procesando en un momento dado. Cada fila de  $Q$  representa un elemento de la secuencia de entrada y encapsula la consulta.<sup>o</sup> la búsqueda de información relevante en el resto de la secuencia que emana de dicho elemento. Imaginemos que cada palabra en una frase es una consulta "buscando entender su contexto.
- **Clave (K):** La matriz de Clave ( $K$ ) actúa como un catálogo de claves.<sup>o</sup> etiquetas descriptivas que identifican las diferentes partes de la secuencia de entrada. Al igual que  $Q$ , cada fila de  $K$  corresponde a un elemento de la secuencia de entrada, pero en este caso, representa la clave "que describe la información contenida en ese elemento y que podría ser relevante para responder a las consultas. Siguiendo con la analogía, cada palabra en la frase también proporciona una clave "que describe su propio significado y rol en la oración.
- **Valor (V):** La matriz de Valor ( $V$ ) almacena la información concreta o el "valor.<sup>a</sup> asociado a cada clave". De nuevo, cada fila de  $V$  se corresponde con un elemento de la secuencia de entrada y contiene la información que realmente se utilizará para construir la salida final, una vez ponderada por la atención. En nuestra analogía, la matriz de "valor" contendría la representación vectorial del significado de cada palabra, lista para ser combinada y ponderada para formar una representación contextualizada de la frase.

La esencia matemática del mecanismo de atención escalonada se encapsula en la Ecuación 1:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

donde:

- $Q \in \mathbb{R}^{n \times d_k}$  representa la matriz de Consulta, con  $n$  siendo la longitud de la secuencia y  $d_k$  la dimensión de las consultas y claves.
- $K \in \mathbb{R}^{m \times d_k}$  es la matriz de Clave, con  $m$  siendo la longitud de la secuencia (que puede ser diferente de  $n$  en algunos casos, como la atención Encoder-Decoder) y  $d_k$  la dimensión de las claves.
- $V \in \mathbb{R}^{m \times d_v}$  denota la matriz de Valor, con  $m$  siendo la longitud de la secuencia de valores y  $d_v$  la dimensión de los valores.
- $d_k$  es el factor de escala, específicamente la raíz cuadrada de la dimensión de las claves ( $d_k$ ), que se utiliza para mitigar el problema del desvanecimiento del gradiente, especialmente con dimensiones elevadas.
- softmax es la función softmax, encargada de normalizar las puntuaciones de similitud, transformándolas en una distribución de probabilidad que representa los pesos de atención.

El proceso computacional que subyace a la atención escalonada se desarrolla en los siguientes pasos:

1. **Cálculo de Similitud (Producto Punto):** Inicialmente, se evalúa la similitud entre cada consulta y cada clave mediante el **producto punto matricial**  $QK^T$ . Este producto punto cuantifica el grado de correspondencia o alineamiento entre las consultas y las claves, generando una matriz de **puntuaciones de similitud**.
2. **Escalado:** Las puntuaciones de similitud obtenidas se someten a un proceso de **escalado**, dividiéndolas por  $\sqrt{d_k}$ . Este paso de escalado es de vital importancia para la **estabilidad del entrenamiento**, ya que previene que las puntuaciones se disparen a valores excesivamente grandes, lo cual podría conducir a **gradientes extremadamente pequeños** tras la aplicación de la función softmax, dificultando el **aprendizaje efectivo**.
3. **Normalización (Softmax):** A continuación, la **función softmax** entra en juego para **normalizar** las puntuaciones escaladas. La softmax transforma estas puntuaciones en **pesos de atención probabilísticos**, que oscilan entre 0 y 1 y suman 1 para cada consulta. Estos **pesos de atención** reflejan la **importancia relativa** de cada valor con respecto a cada consulta, indicando en qué medida cada valor debe contribuir a la representación final.
4. **Ponderación y Agregación (Ponderación por Valores):** Finalmente, los pesos de atención normalizados se utilizan para **ponderar** la matriz de Valor  $V$ . Esta ponderación se materializa multiplicando los pesos de atención por los valores correspondientes y agregando los resultados mediante una **suma ponderada**. La **salida resultante** de este proceso es una matriz que representa la secuencia de entrada, pero ahora **enriquecida con información contextualizada**, donde cada

elemento se ha refinado y ajustado en función de su relevancia con respecto a los demás elementos de la secuencia, guiado por el mecanismo de atención.

### 3.0.3 Estructura de Capas en un Transformer

Un Transformer típico se compone de varias capas de codificación (**Encoder**) y decodificación (**Decoder**). Estas capas se apilan secuencialmente, creando una arquitectura profunda capaz de aprender representaciones complejas de los datos. Cada capa, tanto en el Encoder como en el Decoder, está construida con un conjunto específico de subcapas interconectadas, diseñadas para extraer información relevante y transformar los datos de entrada. Las subcapas fundamentales incluyen la **atención multi-cabeza** (Multi-Head Attention), **redes neuronales feed-forward**, **normalización de capas** (Layer Normalization) y **conexiones residuales**. A continuación, se detalla la estructura y función de cada componente.

#### 3.0.3.1 Capas del Encoder

Cada capa del Encoder está diseñada para procesar la secuencia de entrada y extraer características relevantes. Típicamente, una capa de Encoder consta de las siguientes subcapas principales:

1. **Atención Multi-Cabeza (Multi-Head Attention):** Esta es la subcapa central del Transformer. Permite al modelo ponderar la importancia de diferentes partes de la secuencia de entrada al procesar cada posición. En lugar de una única atención, la atención multi-cabeza utiliza múltiples "cabezas" de atención en paralelo. Cada cabeza aprende a atender a diferentes aspectos de la información, capturando una variedad más rica de relaciones y dependencias dentro de la secuencia. La salida de todas las cabezas se concatena y se proyecta linealmente para producir la salida final de esta subcapa.
2. **Añadir y Normalizar (Add & Norm) - Conexión Residual y Normalización de Capas:** Después de la subcapa de atención multi-cabeza, se aplica una **conexión residual**. Esto significa que la entrada original a la subcapa de atención se suma a la salida de la atención multi-cabeza. Esta conexión residual ayuda a mitigar el problema del desvanecimiento del gradiente en redes profundas, facilitando el entrenamiento de modelos más profundos. A continuación, se aplica la **normalización de capas** (Layer Normalization). La normalización de capas ayuda a estabilizar el aprendizaje normalizando las salidas de las subcapas, lo que acelera el entrenamiento y mejora la generalización.
3. **Red Neuronal Feed-Forward (Feed-Forward Network):** Tras la subcapa de atención y la normalización, la salida se pasa a través de una **red neuronal feed-forward**. Esta red es aplicada de forma idéntica e independiente a cada posición en la secuencia. Consiste típicamente en dos capas lineales con una función de

activación no lineal (como ReLU) entre ellas. La red feed-forward ayuda a procesar la información aprendida por la atención, proporcionando no linealidad y permitiendo al modelo aprender transformaciones más complejas.

4. **Añadir y Normalizar (Add & Norm) - Conexión Residual y Normalización de Capas:** Similar al paso anterior, se aplica otra conexión residual y normalización de capas después de la red neuronal feed-forward. La entrada a la red feed-forward se suma a su salida, y el resultado se normaliza.

### 3.0.3.2 Capas del Decoder

Las capas del Decoder tienen la función de generar la secuencia de salida, utilizando la información codificada por el Encoder. Una capa de Decoder es similar a una capa de Encoder, pero incluye una subcapa de atención adicional para interactuar con la salida del Encoder. Las subcapas típicas de una capa de Decoder son:

1. **Atención Multi-Cabeza Enmascarada (Masked Multi-Head Attention):** Similar a la atención multi-cabeza del Encoder, pero con una modificación crucial: el **enmascaramiento**. En el contexto del Decoder, especialmente en tareas de generación secuencial, es importante que al predecir la salida en una posición dada, el modelo solo pueda atender a las posiciones *anteriores* en la secuencia de salida (ya generada) y no a las posiciones futuras. El enmascaramiento se implementa para evitar que el modelo "vea" la respuesta durante el entrenamiento, asegurando que aprenda a generar la secuencia de forma autoregresiva.
2. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la atención multi-cabeza enmascarada, de la misma manera que en el Encoder.
3. **Atención Encoder-Decoder (Encoder-Decoder Attention o Multi-Head Attention):** Esta subcapa es específica del Decoder y es fundamental para la interacción entre el Encoder y el Decoder. En esta subcapa, el Decoder atiende a la salida del *Encoder*. Las "queries" provienen de la salida de la subcapa de atención enmascarada del Decoder, mientras que las "keys" "values" provienen de la salida del *Encoder*. Esto permite al Decoder enfocar su atención en las partes relevantes de la secuencia de entrada codificada por el Encoder al generar cada elemento de la secuencia de salida.
4. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la atención Encoder-Decoder.
5. **Red Neuronal Feed-Forward (Feed-Forward Network):** Idéntica a la red neuronal feed-forward utilizada en las capas del Encoder.
6. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la red neuronal feed-forward.

### 3.0.3.3 Diagrama Conceptual de un Bloque Transformer

Un diagrama de un bloque Transformer típico mostraría:

- **Para un bloque Encoder:**
  - Un bloque etiquetado *.Entrada* que representa la secuencia de entrada (o la salida de la capa Encoder anterior).
  - Una flecha que lleva a un bloque etiquetado "Multi-Head Attention".
  - Una conexión residual que salta la subcapa "Multi-Head Attention" se suma a la salida de esta subcapa.
  - Un bloque etiquetado *.Add & Norm* después de la suma de la conexión residual.
  - Una flecha desde *.Add & Norm* a un bloque etiquetado "Feed-Forward Network".
  - Otra conexión residual que salta la subcapa "Feed-Forward Network" se suma a la salida de esta subcapa.
  - Un bloque etiquetado *.Add & Norm* después de la segunda suma de la conexión residual, representando la salida de la capa Encoder.
- **Para un bloque Decoder:**
  - Un bloque etiquetado *.Entrada* que representa la secuencia de entrada al Decoder (o la salida de la capa Decoder anterior).
  - Una flecha que lleva a un bloque etiquetado "Masked Multi-Head Attention".
  - Una conexión residual y *.Add & Norm* después de la atención enmascarada.
  - Una flecha a un bloque etiquetado *.Encoder-Decoder Attention* (o simplemente "Multi-Head Attention" si se entiende el contexto). Este bloque también recibiría una conexión de la salida del Encoder (en un diagrama para la arquitectura completa).
  - Una conexión residual y *.Add & Norm* después de la atención Encoder-Decoder.
  - Una flecha a un bloque etiquetado "Feed-Forward Network".
  - Una conexión residual y *.Add & Norm* después de la red feed-forward, representando la salida de la capa Decoder.

En resumen, la estructura de capas de un Transformer, con sus componentes de atención multi-cabeza, redes feed-forward, normalización de capas y conexiones residuales, proporciona un marco robusto y eficiente para el modelado de secuencias complejas, tanto en tareas de codificación como de decodificación. La atención multi-cabeza es el mecanismo central que permite al modelo capturar dependencias ricas, mientras que las otras subcapas facilitan el entrenamiento y mejoran la capacidad de aprendizaje de la red.

### 3.0.4 Retropropagación en Modelos Transformer

El entrenamiento de los modelos Transformer se realiza mediante el algoritmo de **retropropagación** (Backpropagation) <sup>?</sup>, que ajusta los pesos del modelo iterativamente para minimizar una función de pérdida. En esencia, la retropropagación es un método para calcular eficientemente el gradiente de la función de pérdida con respecto a los parámetros del modelo (pesos y biases). Este gradiente indica la dirección en la que se deben ajustar los parámetros para reducir la pérdida y, por lo tanto, mejorar el rendimiento del modelo.

El proceso de retropropagación en un Transformer sigue los principios generales de este algoritmo, pero con consideraciones específicas para las capas de atención y las redes feed-forward que componen su arquitectura. A grandes rasgos, el proceso se puede describir en los siguientes pasos:

1. **Paso Forward (Paso hacia adelante):** Se propaga la secuencia de entrada a través de la red Transformer, desde la capa de entrada hasta la capa de salida. En este paso, se calculan las salidas de cada subcapa (atención multi-cabeza, redes feed-forward, normalización, etc.) secuencialmente. Específicamente, en la subcapa de atención, se calculan las matrices de Consulta (Q), Clave (K) y Valor (V), se realiza el producto punto escalado, se aplica la función Softmax para obtener los pesos de atención y finalmente se calcula la salida ponderada por los valores. Las redes feed-forward aplican transformaciones lineales y no lineales a la salida de la atención.
2. **Cálculo de la Pérdida:** Una vez obtenida la salida del modelo para la secuencia de entrada, se compara con la salida deseada (la "verdad ground") utilizando una función de pérdida adecuada para la tarea en cuestión (e.g., cross-entropy para clasificación, error cuadrático medio para regresión). Esta función de pérdida cuantifica la discrepancia entre la predicción del modelo y el valor real.
3. **Paso Backward (Paso hacia atrás):** Se inicia el proceso de retropropagación propiamente dicho. El gradiente de la función de pérdida se calcula con respecto a la salida del modelo. Luego, utilizando la **regla de la cadena del cálculo diferencial**, este gradiente se propaga hacia atrás a través de cada capa de la red, desde la capa de salida hasta la capa de entrada.
  - **Retropropagación a través de la Atención Multi-Cabeza:** La retropropagación a través de la atención es un poco más compleja que en las capas lineales tradicionales. El gradiente se propaga a través de la función Softmax, el escalado, el producto punto, y de vuelta a través de las matrices de Consulta, Clave y Valor. Esto implica calcular los gradientes con respecto a los pesos de proyección que generaron Q, K y V, así como con respecto a las entradas a la capa de atención. Es importante tener en cuenta que el gradiente también se propaga a través de los pesos de atención, afectando

a todas las partes de la secuencia de entrada que contribuyeron a la salida actual.

- **Retropropagación a través de Redes Feed-Forward:** La retropropagación a través de las redes feed-forward es similar a la de las redes neuronales multicapa estándar. El gradiente se propaga a través de la función de activación no lineal (e.g., ReLU) y las capas lineales, calculando los gradientes con respecto a los pesos y biases de estas capas.
  - **Retropropagación a través de Add & Norm y Conexiones Residuales:** La retropropagación a través de las **conexiones residuales** es sencilla: el gradiente se divide y se suma tanto al camino principal como al camino de la conexión residual. En las capas de normalización, la retropropagación implica calcular los gradientes a través de las operaciones de normalización (media, varianza) y los parámetros aprendibles de la normalización (si los hay, como en Layer Normalization con parámetros gamma y beta).
4. **Actualización de Pesos:** Una vez calculados los gradientes para todos los parámetros del modelo, se utilizan para actualizar los pesos y biases en la dirección opuesta al gradiente, utilizando un algoritmo de optimización como el descenso de gradiente (Gradient Descent) o sus variantes (e.g., Adam, SGD con momentum). La tasa de aprendizaje (learning rate) controla el tamaño del paso en la dirección del gradiente.

#### 3.0.4.1 Consideraciones Específicas para Transformers en Retropropagación

- **Costo Computacional de la Atención:** El cálculo de la atención, especialmente en secuencias largas, puede ser computacionalmente costoso tanto en el paso forward como en el backward. La complejidad de la atención escalonada es cuadrática con respecto a la longitud de la secuencia ( $O(L^2)$ , donde  $L$  es la longitud de la secuencia). Esto puede ser un cuello de botella durante el entrenamiento, especialmente para secuencias muy largas. Técnicas como la atención dispersa (sparse attention) o la atención lineal se han propuesto para mitigar este problema.
- **Estabilidad del Gradiente:** Las conexiones residuales y la normalización de capas en los Transformers ayudan a mejorar la estabilidad del gradiente durante la retropropagación, facilitando el entrenamiento de redes profundas y reduciendo problemas como el desvanecimiento o la explosión del gradiente.
- **Paralelización:** Aunque el mecanismo de atención permite la paralelización en el paso forward, la retropropagación también se puede paralelizar en gran medida, especialmente en hardware como GPUs, lo que acelera significativamente el entrenamiento de los Transformers.

En resumen, la retropropagación en los modelos Transformer es una aplicación del algoritmo estándar, adaptada a la estructura específica de la red, con especial atención al cálculo de gradientes a través del mecanismo de atención. La eficiencia y

estabilidad del entrenamiento se ven favorecidas por las características arquitectónicas de los Transformers, como la atención multi-cabeza, las conexiones residuales y la normalización de capas.

### 3.1 SELF-ATTENTION-BASED IMPUTATION FOR TIME SERIES (SAITS)

SAITS (Self-Attention-based Imputation for Time Series) es un modelo Transformer específicamente diseñado para la imputación de datos faltantes en series temporales. Aprovecha el mecanismo de auto-atención para capturar dependencias temporales y espaciales en los datos de series temporales, lo que permite una imputación más precisa en comparación con los métodos tradicionales. A diferencia de los modelos de imputación clásicos que a menudo se basan en suposiciones simplificadas sobre la estructura temporal de los datos, SAITS aprende directamente estas dependencias de los propios datos, ofreciendo una solución más flexible y robusta.

#### 3.1.1 *Arquitectura del Modelo SAITS*

SAITS adapta la arquitectura Transformer estándar para la tarea de imputación de series temporales, principalmente utilizando una estructura de **Encoder Transformer**. Dado que el objetivo principal es la imputación y no la generación secuencial, la arquitectura Decoder no es estrictamente necesaria en este contexto, aunque variaciones podrían incluirla. Las adaptaciones clave en la arquitectura de SAITS para la imputación de series temporales son:

- **Entrada de Series Temporales:** SAITS toma como entrada series temporales con posibles valores faltantes. Típicamente, la serie temporal se representa como una secuencia de vectores, donde cada vector representa un punto de tiempo y contiene las mediciones de diferentes variables en ese instante. Para manejar los valores faltantes, se pueden emplear **máscaras de entrada** que indican las posiciones donde los datos son desconocidos. Estas máscaras pueden ser utilizadas por el modelo para enfocar la atención en las partes observadas de la serie temporal durante la imputación.
- **Encoder Transformer para Imputación:** SAITS utiliza un stack de capas Encoder Transformer. Cada capa Encoder, como se describió anteriormente, contiene subcapas de atención multi-cabeza, redes feed-forward, normalización de capas y conexiones residuales. Estas capas procesan la serie temporal de entrada para aprender representaciones latentes que capturan las dependencias temporales y las relaciones entre las variables. La **auto-atención** dentro del Encoder permite que cada punto de tiempo en la serie temporal atienda a todos los demás puntos, ponderando su importancia relativa para la imputación en la posición actual.



- **Codificación Posicional Adaptada (Opcional):** Si bien la codificación posicional es crucial en NLP para indicar el orden secuencial de las palabras, su uso en series temporales puede ser adaptado. En algunos casos, la posición temporal relativa ya está implícita en la secuencia de entrada. Sin embargo, se pueden utilizar codificaciones posicionales para explicitar la posición temporal y potencialmente codificar información adicional como el tiempo transcurrido desde el inicio de la serie o patrones temporales cíclicos (e.g., día de la semana, mes del año). La decisión de incluir o adaptar la codificación posicional puede depender de las características específicas del conjunto de datos de series temporales.
- **Capa de Salida para Imputación:** La salida del Encoder Transformer, tras pasar por las capas Encoder, se utiliza para generar los valores imputados. Típicamente, esto se realiza mediante una **capa lineal** final que proyecta la representación latente de cada punto de tiempo a la dimensión del espacio de características original. Esta capa lineal aprende a mapear las representaciones aprendidas por el Transformer a los valores imputados para cada variable en cada punto de tiempo donde falta información.

### 3.1.2 Función de Pérdida para SAITS

La función de pérdida comúnmente utilizada para entrenar SAITS en tareas de imputación es el **Error Cuadrático Medio (MSE)** o variantes del mismo, aplicadas **solo a las posiciones donde los datos estaban originalmente faltantes**. Esto se debe a que el objetivo es minimizar el error en la predicción de los valores faltantes, sin penalizar al modelo por los valores ya observados. Formalmente, si  $Y$  es la serie temporal original (con valores faltantes),  $\hat{Y}$  es la serie temporal imputada por SAITS y  $M$  es una matriz máscara binaria donde  $M_{ij} = 1$  si el valor  $Y_{ij}$  estaba faltante y  $M_{ij} = 0$  si estaba observado, la función de pérdida MSE puede definirse como:

$$\mathcal{L}_{MSE} = \frac{1}{\sum_{i,j} M_{ij}} \sum_{i,j} M_{ij} (Y_{ij} - \hat{Y}_{ij})^2 \quad (2)$$

Esta función de pérdida asegura que el modelo se centre en aprender a imputar los valores faltantes con precisión, ignorando el error en las posiciones donde ya se disponía de información. Otras funciones de pérdida robustas, como el Mean Absolute Error (MAE) o funciones de pérdida basadas en cuantiles, también podrían ser consideradas dependiendo de las características de los datos y los objetivos específicos de la imputación.

### 3.1.3 *Ventajas de Self-Attention para Imputación de Series Temporales*

El uso del mecanismo de auto-atención en SAITS ofrece varias ventajas significativas para la imputación de series temporales en comparación con métodos tradicionales:

- **Modelado de Dependencias a Largo Plazo:** La auto-atención permite a SAITS capturar dependencias temporales a largo plazo de manera efectiva. A diferencia de métodos como ARIMA o RNNs que pueden tener dificultades para recordar información a través de largos periodos de tiempo, la atención permite a SAITS acceder directamente a cualquier punto en la serie temporal al imputar un valor, sin importar la distancia temporal. Esto es crucial para series temporales con patrones complejos y dependencias que se extienden a lo largo del tiempo.
- **Captura de Dependencias Espaciales y Temporales Conjuntas:** En series temporales multivariadas, donde se miden múltiples variables a lo largo del tiempo, la auto-atención puede capturar tanto las dependencias temporales dentro de cada variable como las dependencias espaciales (correlaciones) entre diferentes variables en diferentes puntos de tiempo. El modelo aprende a imputar un valor faltante considerando la información de todas las variables en todos los puntos de tiempo relevantes.
- **Flexibilidad y Adaptabilidad:** La arquitectura Transformer es inherentemente flexible y adaptable a diferentes tipos de series temporales y patrones de datos faltantes. SAITS no requiere suposiciones fuertes sobre la estacionariedad de la serie temporal o la naturaleza de las dependencias temporales. Puede aprender patrones complejos directamente de los datos y adaptarse a diferentes tipos de series temporales sin necesidad de ingeniería de características manual extensiva.
- **Robustez ante Datos Irregulares y Valores Atípicos:** La atención puede ayudar a SAITS a ser más robusto ante datos irregulares y valores atípicos en las series temporales. Al ponderar la importancia de diferentes partes de la secuencia, la atención puede dar menos peso a los valores atípicos o a las regiones ruidosas de la serie temporal durante la imputación, enfocándose en la información más consistente y relevante.

En resumen, SAITS representa un avance significativo en la imputación de series temporales al aplicar la poderosa arquitectura Transformer y el mecanismo de auto-atención. Su capacidad para modelar dependencias complejas, su flexibilidad y robustez lo convierten en una herramienta valiosa para el análisis de datos de series temporales con valores faltantes en diversos dominios.

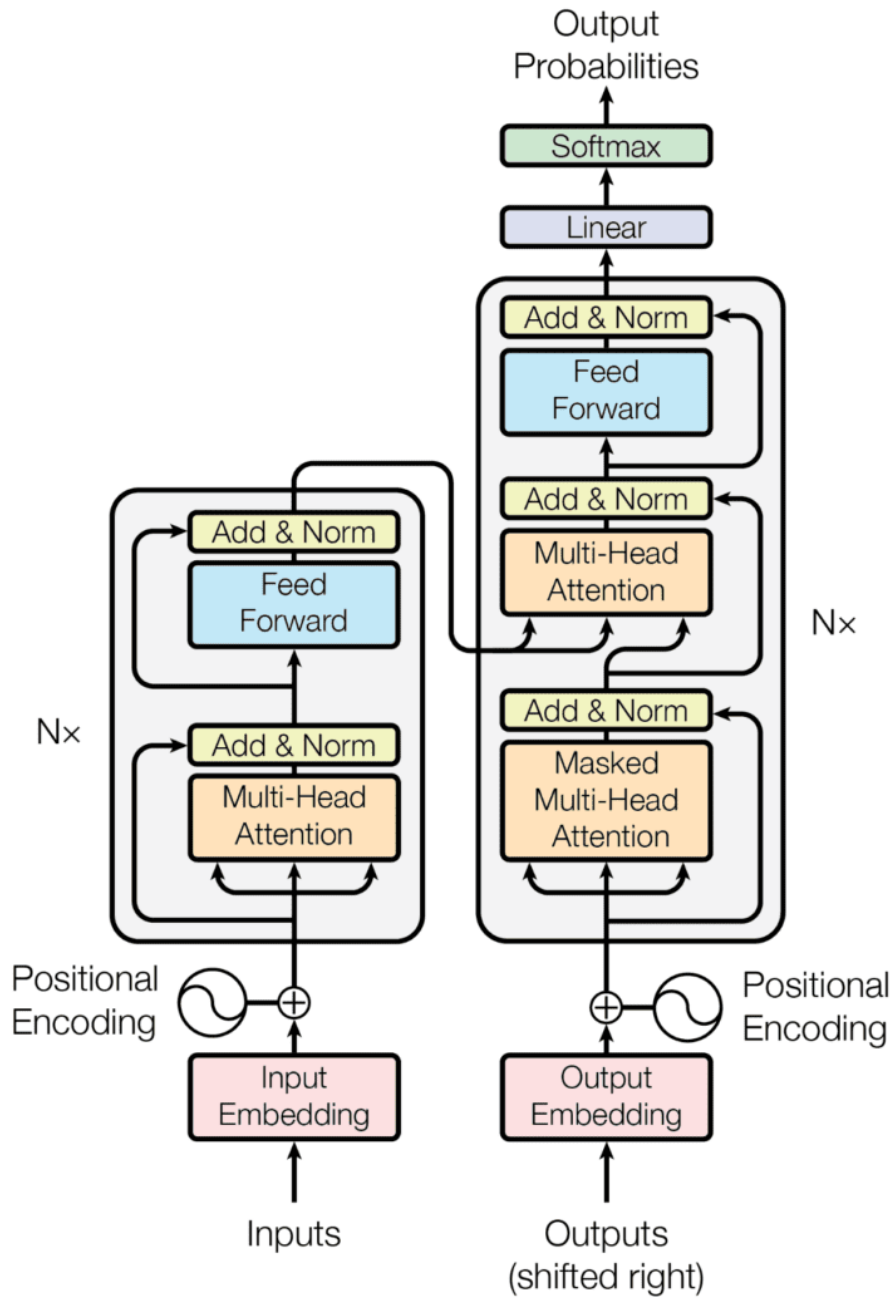


Figura 1: Diagrama conceptual de un bloque Transformer típico (Encoder y Decoder).

---

## ANÁLISIS EXPLORATORIO DE LOS DATOS Y PIPELINE DE PREPROCESAMIENTO E IMPUTACIÓN

---

### TAXONOMÍA DE IMPUTACIÓN PARA VALORES FALTANTES EN SERIES TEMPORALES: ENFOQUES DE MODELOS LOCALES, ESTACIONARIOS Y PREDICTIVOS

Los valores faltantes en series temporales representan un desafío significativo en diversas disciplinas ???. La imputación, el proceso de rellenar estos valores, es crucial para análisis posteriores como la predicción y detección de anomalías ?. Este trabajo presenta una taxonomía de imputación dividida en tres categorías principales: Imputación con Información Local, Imputación Estacionaria e Imputación con Modelos Predictivos.

La Imputación con Información Local utiliza puntos de datos temporalmente cercanos, asumiendo similitud temporal ?. Técnicas comunes incluyen LOCF/NOCB, interpolación lineal y spline, promedios móviles e imputación por vecinos más cercanos ??. Estos métodos son simples y útiles para brechas cortas, pero pueden introducir sesgos ?.

La Imputación Estacionaria se basa en la estacionariedad de la serie temporal, reemplazando valores faltantes con estadísticas como la media, mediana, moda o medias estacionales ??. El algoritmo Expectation-Maximization (EM) también puede aplicarse bajo este supuesto ?. Estos métodos son sencillos pero asumen propiedades estadísticas constantes y pueden sesgar si la estacionariedad no se cumple ?.

La Imputación con Modelos Predictivos emplea modelos estadísticos o de aprendizaje automático para predecir valores faltantes, abarcando desde regresión lineal hasta redes neuronales profundas como RNNs y Transformers ??. Estos modelos capturan dependencias temporales complejas y son más adaptables, aunque requieren mayor complejidad computacional y validación cuidadosa para evitar el sobreajuste ?.

La selección del método de imputación depende de las características de la serie temporal (estacionariedad, estacionalidad, ruido), la naturaleza de los datos faltantes y los objetivos del análisis ?. Las tendencias emergentes incluyen métodos híbridos, avances en aprendizaje profundo, imputación probabilística e integración de la impu-

tación con tareas posteriores ????. Los desafíos abiertos persisten en el manejo de datos MNAR, la evaluación de la calidad de la imputación sin verdad fundamental y la necesidad de conjuntos de datos de referencia estandarizados ????. La investigación futura se dirige a modelos generativos profundos, métodos robustos para datos MNAR e imputación adaptativa ??.

En conclusión, la elección del método de imputación en series temporales requiere una evaluación cuidadosa de las características de los datos y los objetivos del análisis, con un campo en evolución constante hacia técnicas más avanzadas y adaptativas.

#### 4.1 PREPROCESAMIENTO E IMPUTACIÓN

En esta sección, se detallan los pasos de preprocesamiento...

##### 4.1.1 *Imputación por Mediana*

Se realizó una imputación inicial utilizando la mediana...

##### 4.1.2 *Imputación con SAITS*

Posteriormente, se aplicó el modelo SAITS...

---

## CONCLUSIONES Y TRABAJO FUTURO

---

### 5.1 CONCLUSIONES

#### 5.1.1 *Principales Contribuciones del Trabajo*

#### 5.1.2 *Impacto del Uso de Datos Sintéticos en IoT*

### 5.2 TRABAJO FUTURO

#### 5.2.1 *Mejora de los Modelos Transformer*

#### 5.2.2 *Extensiones a Otros Sistemas IoT*

#### 5.2.3 *Posibles Aplicaciones en Diferentes Dominios*

## APPENDIX A: SUPPLEMENTARY MATERIAL

---

# **pampaneira\_imputation**

***Release 0.1.0***

**Ricardo Ruiz**

**Apr 07, 2025**





## CONTENTS:

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>pampaneira_imputation package</b>            | <b>3</b>  |
| 1.1      | Submodules                                      | 3         |
| 1.2      | pampaneira_imputation.config module             | 3         |
| 1.3      | pampaneira_imputation.data_loader module        | 3         |
| 1.4      | pampaneira_imputation.data_preprocessor module  | 5         |
| 1.5      | pampaneira_imputation.evaluation module         | 9         |
| 1.6      | pampaneira_imputation.imputation_methods module | 9         |
| 1.7      | pampaneira_imputation.utils module              | 11        |
| 1.8      | Module contents                                 | 12        |
| <b>2</b> | <b>tests package</b>                            | <b>13</b> |
| 2.1      | Submodules                                      | 13        |
| 2.2      | tests.conftest module                           | 13        |
| 2.3      | tests.minimal_impute_function module            | 13        |
| 2.4      | tests.minimal_test module                       | 13        |
| 2.5      | tests.test_data_preprocessor module             | 13        |
| 2.6      | tests.test_evaluation module                    | 13        |
| 2.7      | tests.test_function_standalone module           | 13        |
| 2.8      | tests.test_imputation_methods module            | 13        |
| 2.9      | tests.test_utils module                         | 13        |
| 2.10     | Module contents                                 | 13        |
|          | <b>Python Module Index</b>                      | <b>15</b> |
|          | <b>Index</b>                                    | <b>17</b> |



Add your content using `reStructuredText` syntax. See the [reStructuredText](#) documentation for details.



## PAMPANEIRA\_IMPUTATION PACKAGE

### 1.1 Submodules

### 1.2 pampaneira\_imputation.config module

### 1.3 pampaneira\_imputation.data\_loader module

```
pampaneira_imputation.data_loader.load_intersection_data(filepath: str =  
                                                         '../data/trafico_contamina_intersección.csv',  
                                                         date_col_original: str = 'Date',  
                                                         date_col_target: str = 'date',  
                                                         truck_pos_col: str = 'truck_pos',  
                                                         target_truck_pos: str = 'PAM_2',  
                                                         timezone: str = 'UTC') → DataFrame
```

Carga datos de intersección, filtra por posición de camión, convierte la fecha.

**Args:**

filepath (str, optional): Ruta al archivo CSV de intersección (por defecto: config.INTERSECTION\_FILE).  
date\_col\_original (str, optional): Nombre original de la columna de fecha en el CSV (por defecto: "Date").  
date\_col\_target (str, optional): Nombre objetivo de la columna de fecha (por defecto: config.DATE\_COL).  
truck\_pos\_col (str, optional): Nombre de la columna de posición del camión (por defecto: config.TRUCK\_POS\_COL).  
target\_truck\_pos (str, optional): Posición objetivo del camión para filtrar (por defecto: config.TARGET\_TRUCK\_POS).  
timezone (str, optional): Zona horaria para las fechas (por defecto: config.TIMEZONE).

**Returns:**

pd.DataFrame: DataFrame con datos de intersección cargados, filtrados y preprocesados.

**Raises:**

FileNotFoundError: Si el archivo especificado no se encuentra. KeyError: Si una columna especificada no se encuentra o falla el cambio de nombre en el archivo.

```
pampaneira_imputation.data_loader.load_traffic_data(filepath: str = '../data/trafico_feb22_ago23.csv',
columns_to_use: List[str] =
['vehicles_PAM_1_OUT',
'vehicles_PAM_1_OUT_Zona_Granada', 'vehic-
cles_PAM_1_OUT_Zona_Catalunia_y_Otras',
'vehi-
cles_PAM_1_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_1_OUT_Extranjero', 'vehic-
cles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_1_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_OUT_Zona_Otras',
'vehicles_PAM_1_OUT_5_seats',
'vehicles_PAM_1_OUT_+5_seats',
'vehicles_PAM_1_OUT_-5_seats',
'vehicles_PAM_1_OUT_nan_seats',
'vehicles_PAM_1_OUT_101-200_CO2',
'vehicles_PAM_1_OUT_0-100_CO2',
'vehicles_PAM_1_OUT_nan_CO2',
'vehicles_PAM_1_OUT_201-300_CO2',
'vehicles_PAM_1_OUT_+300_CO2',
'vehicles_PAM_1_IN',
'vehicles_PAM_1_IN_Zona_Granada',
'vehicles_PAM_1_IN_Zona_Catalunia_y_Otras',
'vehicles_PAM_1_IN_Zona_Andalucia_no_GR',
'vehicles_PAM_1_IN_Extranjero', 'vehic-
cles_PAM_1_IN_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_1_IN_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_IN_Zona_Otras',
'vehicles_PAM_1_IN_5_seats',
'vehicles_PAM_1_IN_+5_seats',
'vehicles_PAM_1_IN_-5_seats',
'vehicles_PAM_1_IN_nan_seats',
'vehicles_PAM_1_IN_101-200_CO2',
'vehicles_PAM_1_IN_0-100_CO2',
'vehicles_PAM_1_IN_nan_CO2',
'vehicles_PAM_1_IN_201-300_CO2',
'vehicles_PAM_1_IN_+300_CO2',
'vehicles_PAM_2_OUT',
'vehicles_PAM_2_OUT_Zona_Granada', 'vehic-
cles_PAM_2_OUT_Zona_Catalunia_y_Otras',
'vehi-
cles_PAM_2_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_2_OUT_Extranjero', 'vehic-
cles_PAM_2_OUT_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_2_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_2_OUT_Zona_Otras',
'vehicles_PAM_2_OUT_5_seats',
'vehicles_PAM_2_OUT_+5_seats',
'vehicles_PAM_2_OUT_-5_seats',
'vehicles_PAM_2_OUT_nan_seats',
'vehicles_PAM_2_OUT_101-200_CO2',
'vehicles_PAM_2_OUT_0-100_CO2',
'vehicles_PAM_2_OUT_nan_CO2',
'vehicles_PAM_2_OUT_201-300_CO2',
'vehicles_PAM_2_OUT_+300_CO2',
'vehicles_PAM_2_IN',
'vehicles_PAM_2_IN_Zona_Granada',
```

Carga datos de tráfico generales, selecciona columnas relevantes, convierte la fecha.

**Args:**

filepath (str, optional): Ruta al archivo CSV de tráfico (por defecto: config.TRAFFIC\_FILE). columns\_to\_use (List[str], optional): Lista de columnas de tráfico a usar (por defecto: config.PAM\_BUB\_TRAFFIC\_COLS). date\_col (str, optional): Nombre de la columna de fecha (por defecto: config.DATE\_COL). timezone (str, optional): Zona horaria para las fechas (por defecto: config.TIMEZONE).

**Returns:**

pd.DataFrame: DataFrame con datos de tráfico cargados y preprocesados.

**Raises:**

FileNotFoundError: Si el archivo especificado no se encuentra. KeyError: Si una columna especificada no se encuentra en el archivo.

## 1.4 pampaneira\_imputation.data\_preprocessor module

`pampaneira_imputation.data_preprocessor.add_period1_padding(df_period1: DataFrame, start_pad: Timestamp = Timestamp('2023-01-17 00:00:00+0000', tz='UTC'), end_pad: Timestamp = Timestamp('2023-03-14 23:00:00+0000', tz='UTC'), freq: str = 'h') → DataFrame`

Añade relleno de NaNs al inicio y al final de los datos del Periodo 1.

**Args:**

df\_period1 (pd.DataFrame): DataFrame del Periodo 1 con DatetimeIndex. start\_pad (pd.Timestamp, optional): Fecha de inicio para el relleno inicial

(por defecto: config.PERIOD\_1\_PADDING\_START).

**end\_pad (pd.Timestamp, optional): Fecha de fin para el relleno final**

(por defecto: config.PERIOD\_1\_PADDING\_END).

**freq (str, optional): Frecuencia para el rango de fechas de relleno**

(por defecto: 'h').

**Returns:**

**pd.DataFrame: DataFrame del Periodo 1 con relleno de NaNs añadido al inicio y al final.**

`pampaneira_imputation.data_preprocessor.fill_missing_timestamps(df: DataFrame, start_date: Timestamp, end_date: Timestamp, freq: str = 'h', date_col: str = 'date') → DataFrame`

Rellena las marcas de tiempo horarias faltantes en un DataFrame con NaNs.

**Args:**

df (pd.DataFrame): DataFrame de entrada con una columna de fecha. start\_date (pd.Timestamp): Fecha de inicio del rango completo. end\_date (pd.Timestamp): Fecha de fin del rango completo. freq (str, optional): Frecuencia para el rango de fechas (por defecto: 'h'). date\_col (str, optional): Nombre de la columna de fecha (por defecto: config.DATE\_COL).

**Returns:**



**pd.DataFrame:** DataFrame con marcas de tiempo horarias completas y NaNs para los datos faltantes.

```

pampaneira_imputation.data_preprocessor.preprocess_for_imputation(df: DataFrame, feature_cols:
    list =
    ['vehicles_PAM_1_OUT',
     'vehic-
     cles_PAM_1_OUT_Zona_Granada',
     'vehic-
     cles_PAM_1_OUT_Zona_Catalunia_y_Otras',
     'vehic-
     cles_PAM_1_OUT_Zona_Andalucia_no_GR',
     'vehic-
     cles_PAM_1_OUT_Extranjero',
     'vehic-
     cles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
     'vehic-
     cles_PAM_1_OUT_Zona_Extremadura_y_Otras',
     'vehic-
     cles_PAM_1_OUT_Zona_Otras',
     'vehic-
     cles_PAM_1_OUT_5_seats',
     'vehic-
     cles_PAM_1_OUT_+5_seats',
     'vehicles_PAM_1_OUT_-
     5_seats',
     'vehic-
     cles_PAM_1_OUT_nan_seats',
     'vehicles_PAM_1_OUT_101-
     200_CO2',
     'vehicles_PAM_1_OUT_0-
     100_CO2',
     'vehic-
     cles_PAM_1_OUT_nan_CO2',
     'vehicles_PAM_1_OUT_201-
     300_CO2',
     'vehic-
     cles_PAM_1_OUT_+300_CO2',
     'vehicles_PAM_1_IN', 'vehic-
     cles_PAM_1_IN_Zona_Granada',
     'vehic-
     cles_PAM_1_IN_Zona_Catalunia_y_Otras',
     'vehic-
     cles_PAM_1_IN_Zona_Andalucia_no_GR',
     'vehic-
     cles_PAM_1_IN_Extranjero',
     'vehic-
     cles_PAM_1_IN_Zona_Comunidad_de_Madrid',
     'vehic-
     cles_PAM_1_IN_Zona_Extremadura_y_Otras',
     'vehic-
     cles_PAM_1_IN_Zona_Otras',
     'vehic-
     cles_PAM_1_IN_5_seats',
     'vehic-
     cles_PAM_1_IN_+5_seats',
     'vehicles_PAM_1_IN_-
     5_seats',
     'vehic-
     cles_PAM_1_IN_nan_seats', 7
     'vehicles_PAM_1_IN_101-
     200_CO2',
     'vehicles_PAM_1_IN_0-

```

#### 1.4. pampaneira\_imputation.data\_preprocessor module

Prepara los datos para modelos de imputación: divide, escala, ventanas, añade datos faltantes.

**Args:**

**df** (pd.DataFrame): DataFrame con DatetimeIndex y columnas de características. **feature\_cols** (list, optional): Lista de columnas a usar como características

(por defecto: config.FEATURE\_COLUMNS).

**train\_start** (str, optional): Fecha de inicio del conjunto de entrenamiento

(por defecto: config.TRAIN\_START\_DATE).

**train\_end** (str, optional): Fecha de fin del conjunto de entrenamiento

(por defecto: config.TRAIN\_END\_DATE).

**val\_start** (str, optional): Fecha de inicio del conjunto de validación

(por defecto: config.VAL\_START\_DATE).

**val\_end** (str, optional): Fecha de fin del conjunto de validación

(por defecto: config.VAL\_END\_DATE).

**test\_start** (str, optional): Fecha de inicio del conjunto de prueba

(por defecto: config.TEST\_START\_DATE).

**test\_end** (str, optional): Fecha de fin del conjunto de prueba

(por defecto: config.TEST\_END\_DATE).

**n\_steps** (int, optional): Tamaño de la ventana deslizante

(por defecto: config.N\_STEPS).

**missing\_rate** (float, optional): Proporción de valores a convertir en NaN

(0 para desactivar) (por defecto: config.MISSING\_RATE).

**missing\_pattern** (str, optional): Patrón de datos faltantes: 'point', 'subseq'

o 'block' (por defecto: config.MISSING\_PATTERN).

**\*\*missingness\_kwargs**: Argumentos adicionales para create\_missingness

(ej., min/max\_length).

**Returns:**

**Dict**: Un diccionario que contiene divisiones de datos procesados

(train\_X, val\_X, test\_X), datos originales (sin máscara) (train\_X\_ori, etc.), el escalador, número de características, y número de pasos. También incluye máscaras indicadoras.

pampaneira\_imputation.data\_preprocessor.split\_by\_period(df: DataFrame, period\_1\_start: Timestamp = Timestamp('2023-01-17 17:00:00+0000', tz='UTC'), period\_1\_end: Timestamp = Timestamp('2023-03-14 11:00:00+0000', tz='UTC'), period\_2\_start: Timestamp = Timestamp('2023-06-06 13:00:00+0000', tz='UTC'), period\_2\_end: Timestamp = Timestamp('2023-06-27 00:00:00+0000', tz='UTC'), date\_col: str = 'date') → Tuple[DataFrame, DataFrame]

Divide el DataFrame en dos periodos basados en fechas predefinidas.

**Args:**

**df** (pd.DataFrame): DataFrame de entrada con una columna de fecha

o DatetimeIndex.

**period\_1\_start (pd.Timestamp, optional):** Fecha de inicio del Periodo 1  
(por defecto: config.PERIOD\_1\_START).

**period\_1\_end (pd.Timestamp, optional):** Fecha de fin del Periodo 1  
(por defecto: config.PERIOD\_1\_END).

**period\_2\_start (pd.Timestamp, optional):** Fecha de inicio del Periodo 2  
(por defecto: config.PERIOD\_2\_START).

**period\_2\_end (pd.Timestamp, optional):** Fecha de fin del Periodo 2  
(por defecto: config.PERIOD\_2\_END).

**date\_col (str, optional):** Nombre de la columna de fecha si no se usa el índice  
(por defecto: config.DATE\_COL).

**Returns:**

**Tuple[pd.DataFrame, pd.DataFrame]:** Tupla que contiene el DataFrame del Periodo 1 y el DataFrame del Periodo 2.

## 1.5 pampaneira\_imputation.evaluation module

**pampaneira\_imputation.evaluation.calculate\_imputation\_metrics**(*y\_true: ndarray, y\_pred: ndarray, indicating\_mask: ndarray*) → Dict[str, float]

Calcula MAE, MSE, RMSE, MRE para valores imputados donde la máscara es 1.

**Args:**

*y\_true* (np.ndarray): Datos verdaderos (potencialmente con NaNs donde faltaban originalmente). *y\_pred* (np.ndarray): Datos imputados. *indicating\_mask* (np.ndarray): Máscara donde 1 indica un valor faltante que fue imputado,

0 indica un valor observado.

**Returns:**

Dict[str, float]: Diccionario que contiene 'mae', 'mse', 'rmse', 'mre'.

**pampaneira\_imputation.evaluation.evaluate\_all\_methods**(*preprocessed\_data: Dict, imputed\_results: Dict[str, ndarray], methods\_to\_evaluate: list = ['median', 'mean', 'linear', 'ffill\_bfill', 'bfill\_ffill', 'saits']*) → DataFrame

Evalúa múltiples métodos de imputación usando los resultados del conjunto de prueba.

**Args:**

*preprocessed\_data* (Dict): Diccionario de preprocess\_for\_imputation. *imputed\_results* (Dict[str, np.ndarray]): Diccionario que mapea nombres de métodos a arrays NumPy imputados (conjunto de prueba). *methods\_to\_evaluate* (list, optional): Lista de claves en *imputed\_results* para evaluar.

(por defecto: ['median', 'mean', 'linear', 'ffill\_bfill', 'bfill\_ffill', 'saits'])

**Returns:**

pd.DataFrame: DataFrame de Pandas que resume MAE, MSE, RMSE, MRE para cada método.

## 1.6 pampaneira\_imputation.imputation\_methods module

**pampaneira\_imputation.imputation\_methods.impute\_forward\_backward**(*X\_missing: ndarray*) → Tuple[ndarray, ndarray]

Realiza la imputación forward-fill y backward-fill en un array 3D.

Aplica forward-fill seguido de backward-fill, y backward-fill seguido de forward-fill, para imputar valores faltantes en cada muestra del array de entrada.

**Args:**

**X\_missing (np.ndarray):** Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n\_samples, n\_steps, n\_features).

**Returns:**

**Tuple[np.ndarray, np.ndarray]:** Tupla que contiene dos arrays NumPy 3D:

- El primero es el resultado de forward-fill seguido de backward-fill.
- El segundo es el resultado de backward-fill seguido de forward-fill.

`pampaneira_imputation.imputation_methods.impute_linear(X_missing: ndarray) → ndarray`

Realiza la interpolación lineal para imputar valores faltantes en un array 3D.

Aplica la interpolación lineal a lo largo del eje del tiempo (axis=0) para cada muestra en el array de entrada.

**Args:**

**X\_missing (np.ndarray):** Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n\_samples, n\_steps, n\_features).

**Returns:**

np.ndarray: Array NumPy 3D con valores faltantes imputados usando interpolación lineal.

`pampaneira_imputation.imputation_methods.impute_mean(X_missing: ndarray) → ndarray`

Imputa los valores faltantes en un array 3D usando la media por característica.

Calcula la media de cada característica (a través de muestras y pasos de tiempo) y reemplaza los valores NaN con la media calculada para esa característica.

**Args:**

**X\_missing (np.ndarray):** Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n\_samples, n\_steps, n\_features).

**Returns:**

np.ndarray: Array NumPy 3D con valores faltantes imputados.

`pampaneira_imputation.imputation_methods.impute_mean_sample_wise(data)`

Imputa los valores faltantes usando la media de cada muestra (a través del tiempo).

Si toda una característica para una muestra es NaN, usa un valor de reserva.

**Args:**

**data (ndarray):** Array NumPy de forma (n\_samples, n\_timestamps, n\_features)

Datos de entrada con valores NaN para ser imputados.

**Returns:**

**ndarray:** Array NumPy de forma (n\_samples, n\_timestamps, n\_features)

Datos con valores NaN imputados.

`pampaneira_imputation.imputation_methods.impute_median(X_missing: ndarray) → ndarray`

Imputa los valores faltantes en un array 3D usando la mediana por característica.

Calcula la mediana de cada característica (a través de muestras y pasos de tiempo) y reemplaza los valores NaN con la mediana calculada para esa característica.

**Args:**

**X\_missing (np.ndarray):** Array NumPy 3D de entrada con valores faltantes (NaNs).  
Forma: (n\_samples, n\_steps, n\_features).

**Returns:**

np.ndarray: Array NumPy 3D con valores faltantes imputados.

pampaneira\_imputation.imputation\_methods.**impute\_median\_sample\_wise**(data)

Imputa los valores faltantes usando la mediana de cada muestra (a través del tiempo).

Si toda una característica para una muestra es NaN, usa un valor de reserva.

**Args:**

**data (ndarray):** Array NumPy de forma (n\_samples, n\_timestamps, n\_features)  
Datos de entrada con valores NaN para ser imputados.

**Returns:**

**ndarray:** Array NumPy de forma (n\_samples, n\_timestamps, n\_features)  
Datos con valores NaN imputados.

## 1.7 pampaneira\_imputation.utils module

pampaneira\_imputation.utils.**create\_missingness**(data: ndarray, rate: float, pattern: str, \*\*kwargs) → ndarray

Introduce valores faltantes (NaN) en un array NumPy 3D (muestras, pasos, características).

Modifica el array in situ por eficiencia, pero también lo devuelve.

**Args:**

data (np.ndarray): Array NumPy 3D de entrada. Forma: (n\_samples, n\_steps, n\_features). rate (float): Tasa de datos faltantes a introducir (entre 0 y 1). pattern (str): Patrón de datos faltantes: 'point', 'subseq' o 'block'. \*\*kwargs: Argumentos adicionales dependientes del patrón:

- Para 'subseq': min\_missing\_len, max\_missing\_len.
- Para 'block': min\_missing\_len, max\_missing\_len.

**Returns:**

np.ndarray: Array NumPy 3D con valores faltantes introducidos.

pampaneira\_imputation.utils.**reshape\_imputed\_to\_df**(imputed\_data: ndarray, original\_index: DatetimeIndex, columns: List[str], n\_steps: int) → DataFrame

Remodela los datos imputados 3D (muestras, pasos, características) de vuelta a un DataFrame 2D.

Asume que el índice original corresponde al *inicio* de cada ventana. Reconstruye la línea de tiempo completa.

**Args:**

**imputed\_data (np.ndarray):** Array NumPy 3D de datos imputados.  
Forma: (n\_samples, n\_steps, n\_features).

original\_index (pd.DatetimeIndex): DatetimeIndex original correspondiente al conjunto de datos a imputar.  
columns (List[str]): Lista de nombres de columnas para el DataFrame reconstruido. n\_steps (int): Número de pasos de tiempo en cada ventana.

**Returns:**

pd.DataFrame: DataFrame de Pandas reconstruido a partir de los datos imputados.

`pampaneira_imputation.utils.sliding_window(data: ndarray, n_steps: int) → ndarray`

Crea ventanas deslizantes a partir de datos secuenciales.

**Args:**

**data (np.ndarray):** Array NumPy 2D o 3D de datos secuenciales.

Si es 2D, se asume forma (n\_timesteps, n\_features). Si es 3D, se asume forma (n\_samples, n\_timesteps, n\_features).

n\_steps (int): Tamaño de cada ventana deslizante.

**Returns:**

**np.ndarray:** Array NumPy 3D de ventanas deslizantes.

**Forma:** (n\_windows, n\_steps, n\_features) si la entrada era 2D,

o (n\_samples, n\_windows, n\_steps, n\_features) si la entrada era 3D. En este caso, siempre devuelve 3D con forma (n\_windows, n\_steps, n\_features).

## 1.8 Module contents

Módulo de inicialización para el paquete `pampaneira_imputation`.

## TESTS PACKAGE

### 2.1 Submodules

#### 2.2 tests.conftest module

#### 2.3 tests.minimal\_impute\_function module

#### 2.4 tests.minimal\_test module

#### 2.5 tests.test\_data\_preprocessor module

#### 2.6 tests.test\_evaluation module

#### 2.7 tests.test\_function\_standalone module

#### 2.8 tests.test\_imputation\_methods module

#### 2.9 tests.test\_utils module

#### 2.10 Module contents





## PYTHON MODULE INDEX

### p

- `pampaneira_imputation`, [12](#)
- `pampaneira_imputation.config`, [3](#)
- `pampaneira_imputation.data_loader`, [3](#)
- `pampaneira_imputation.data_preprocessor`, [5](#)
- `pampaneira_imputation.evaluation`, [9](#)
- `pampaneira_imputation.imputation_methods`, [9](#)
- `pampaneira_imputation.utils`, [11](#)



## A

`add_period1_padding()` (in module `pampaneira_imputation.data_preprocessor`), 5

## C

`calculate_imputation_metrics()` (in module `pampaneira_imputation.evaluation`), 9

`create_missingness()` (in module `pampaneira_imputation.utils`), 11

## E

`evaluate_all_methods()` (in module `pampaneira_imputation.evaluation`), 9

## F

`fill_missing_timestamps()` (in module `pampaneira_imputation.data_preprocessor`), 5

## I

`impute_forward_backward()` (in module `pampaneira_imputation.imputation_methods`), 9

`impute_linear()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_mean()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_mean_sample_wise()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_median()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_median_sample_wise()` (in module `pampaneira_imputation.imputation_methods`), 11

## L

`load_intersection_data()` (in module `pampaneira_imputation.data_loader`), 3

`load_traffic_data()` (in module `pampaneira_imputation.data_loader`), 3

## M

module

`pampaneira_imputation`, 12

`pampaneira_imputation.config`, 3

`pampaneira_imputation.data_loader`, 3

`pampaneira_imputation.data_preprocessor`, 5

`pampaneira_imputation.evaluation`, 9

`pampaneira_imputation.imputation_methods`, 9

`pampaneira_imputation.utils`, 11

## P

`pampaneira_imputation`  
module, 12

`pampaneira_imputation.config`  
module, 3

`pampaneira_imputation.data_loader`  
module, 3

`pampaneira_imputation.data_preprocessor`  
module, 5

`pampaneira_imputation.evaluation`  
module, 9

`pampaneira_imputation.imputation_methods`  
module, 9

`pampaneira_imputation.utils`  
module, 11

`preprocess_for_imputation()` (in module `pampaneira_imputation.data_preprocessor`), 6

## R

`reshape_imputed_to_df()` (in module `pampaneira_imputation.utils`), 11

## S

`sliding_window()` (in module `pampaneira_imputation.utils`), 11

`split_by_period()` (in module *pam-*  
*paneira\_imputation.data\_preprocessor*),  
8